

Memo: what this CDP can and can't tell sales

1. Surface areas

SURFACE	ATTRIBUTION	COLUMNS IT RESTS ON
UI	Deterministic	<code>raw_segment_pages.user_id</code> , <code>path</code>
SDK	Deterministic for 18 of 31 objects, heuristic for 1 more, impossible to separate from the CLI	<code>raw_experiments.repo_info</code> ; <code>metadata.ci_run_id / ci / source</code> ; <code>raw_datasets.metadata.source</code> ; <code>raw_trace_volume (org × day)</code>
CLI	Impossible for usage; heuristic for intent	<code>raw_cli_wizard</code> (a browser event); <code>raw_api_keys.name</code> containing "cli"
MCP	Impossible for usage; heuristic for intent	<code>raw_docs_mcp_events.action</code>

`repo_info` proves an object was logged from code inside a git repo; a self-declared `metadata.source` like `lambda-prod` only suggests it, so it scores 0.60. That is 19 of 31 objects attributed to code — 18 deterministically, 1 heuristically. Nothing in the 18 files names a client, user-agent or key id, so the CLI — which wraps the SDK — can't be separated from it, and the remaining 12 objects carry neither field and are recorded `unknown`, not SDK. The UI blind spot is the org: a page view has a `user_id` and no `org_id`, so views by a two-account person stay unattributed.

An object's `user_id` is the credential that made the call, not necessarily the person. Bob's 12 nightly runs carry `ci_run_id`, an author of `ci-runner` and a key called `ci-service`, so they're CI: tagged `actor_type = automation`, never counted as a member using the product. That distinction survives all the way to the member view — `dim_members.sdk_evidence` reads `no_evidence` for Bob while `sdk_evidence_any_actor` reads `usage_proven`, so the machine is visible without being mistaken for a user.

2. Resolution quality

Five things, all recomputed by `rpt_resolution_quality` every run. **Precision** on a monthly steward-labelled sample, split by `match_method`, with a bar of 98% for anything a rep sees. **Coverage**: 8 of 11 companies carry an account, though only 7 deliver signals — the eighth is held for review. **Corroboration**: 4 of 8 matches rest on two or more independent keys. **Conflict and ambiguity rates**, watched as drift alarms. And **how often reps click "wrong account"**.

On the trade-off I'd take precision every time. A bad merge sends a rep another company's news and costs the tool its credibility; a miss just sits in the review queue where someone can still find it. That's why:

- only a key unique to one account decides, and LinkedIn beats domain (`hooli.com` belongs to two accounts);
- every other key has to agree, or it's a conflict (`sc11` has Acme's domain and Pied Piper's LinkedIn, so it gets no account);
- a name never decides, and when a name points at a different account the match is held for review (`sc05 "Hooli Inc"` resolving to Hooli XYZ);
- a headline that names a different company never reaches a rep (`n5`).

Unresolved splits into two outputs. A company no account could exist for becomes a net-new prospect; a conflict or a company with no identifiers goes to the review queue, because an account probably *does* exist and calling it "net new" would be a second error on top of the first.

3. "% of the account using the CLI"

No, not truthfully. For Globex I'd give three numbers: CLI usage is *not measurable*; setup intent covers 1 of 3 known product members, with a second not counted because he belongs to two accounts; and code-based usage by a person, which can't be split between SDK and CLI, is 1 of 3, plus a CI job under a member's key. Where the only proof is org-level — Initech's trace ingestion, with no member attributable — `fct_account_surface_adoption.evidence_level` says `org`, so "SDK: `usage_proven`" can never be read as "a person here uses the SDK".

I won't give a CLI percentage, won't infer CLI from `repo_info` or key names, and would drop "% of the account" altogether — the denominator is product members, not headcount. `cli_and_mcp_never_claimed_as_usage` keeps that honest in code, and `account_360.adoption_in_plain_english` carries those exact sentences to the rep.

4. Two changes

- **Product, instrumentation.** *Captured* at the API gateway from the client user-agent or MCP server id: stamp `client_surface`, client version and `api_key_id` on every write. *Stored* denormalised on the object tables and in a new `api_request_log`. CLI and MCP become deterministic, SDK separates from CLI, and service keys identify themselves.
- **CRM, data addition.** *Captured* from an enrichment vendor at account creation and on a refresh schedule. *Stored* as required `website_domain` and `linkedin_company_url` on Account, plus a populated `ParentId`. Missing identifiers cause most non-matches; the hierarchy lets Hooli and Hooli XYZ roll up without merging.

5. Sales motion

This serves expansion: usage surges on non-Enterprise plans, CLI and MCP intent as an enablement opening, and funding, launch or hiring news. `net_new_prospects` also feeds a product-led motion into sales. Two leading indicators tell me whether it's working — do reps open the 360, and what share of signals get acted on rather than dismissed. Two lagging ones settle it: signal-sourced pipeline and expansion ARR, measured against a random holdout of accounts. A signal type with no lift, or more than ~70% dismissed, gets cut.

6. Which signals drive value

1. Fix the outcome first: an opportunity, upgrade or expansion within 30 to 90 days.
2. Snapshot `fct_account_signals` daily. It's already one shape, so it works as the feature store and joins straight to opportunities.
3. Per signal type, measure lift over baseline (controlled for plan and size), lead time and precision at top-N. Then learn weights to replace my v0 rules, keeping the reasons visible.
4. Use holdouts so signals don't take credit for deals that would have closed anyway, treat rep dispositions as labels, and retire whatever shows no lift.